참고 자주 나오는 질문

체인과 Agent는 무엇이 다른가
LCEL 파이프는 함수를 순서대로 부르는 것과 무엇이 다른가
청크 크기는 얼마가 정답인가
검색 결과가 나왔는데 답이 틀리는 이유
임베딩 모델을 중간에 바꾸면 어떻게 되는가
State는 그냥 변수와 무엇이 다른가
Reducer는 왜 필요한가
ReAct 루프가 멈추지 않을 때
도구를 부르지 않고 답을 지어낼 때

체인과 Agent는 무엇이 다른가

체인은 경로가 코드에 적혀 있습니다. 프롬프트를 거쳐 모델을 거쳐 파서로 나가는 순서가 고정입니다. 입력이 무엇이든 같은 길을 지납니다.

Agent는 경로를 모델이 정합니다. 도구를 쓸지, 어떤 도구를 쓸지, 몇 번 쓸지를 실행 중에 판단합니다.

그래서 체인은 결과를 예측할 수 있고 Agent는 예측할 수 없습니다. 예측할 수 없다는 것은 단점이 아니라 미리 모든 경우를 짜 둘 수 없는 일에 쓰라는 뜻입니다. 판단 기준은 하나입니다.

처리 순서를 미리 적을 수 있으면 체인이고 없으면 Agent입니다.

LCEL 파이프는 함수를 순서대로 부르는 것과 무엇이 다른가

하는 일은 같습니다. 다만 파이프로 묶으면 세 가지가 따라옵니다.

`invoke` , `stream` , `batch` 세 가지 실행 방식을 그대로 씁니다. 중간에 어떤 부품을 끼워도 인터페이스가 같습니다. 그리고 `astream_events` 로 안을 들여다볼 수 있습니다.

직접 함수로 이으면 이 셋을 매번 다시 짜야 합니다. 부품을 같은 규격으로 맞춰 두는 것이 목적입니다.

청크 크기는 얼마가 정답인가

정답이 없습니다. 그리고 이것이 이 질문의 답입니다.

작게 자르면 정밀하지만 맥락이 잘립니다. 크게 자르면 맥락은 살지만 한 청크에 여러 주제가 섞여 검색이 흐려집니다. 출발점은 한국어 기준 500자에서 1000자, 겹침은 50자에서 150자입니다. 여기서부터는 감이 아니라 평가셋으로 정합니다.

2일차 도전 미션에서 평가셋을 만든 이유가 이것입니다. 값을 바꿀 때마다 같은 시험지로 채점해 좋아졌는지 나빠졌는지를 봅니다.

검색 결과가 나왔는데 답이 틀리는 이유

벡터 검색은 항상 k개를 돌려줍니다. 관련 문서가 하나도 없어도 그중 덜 먼 것을 k개 골라 줍니다.

결과가 나왔다는 것과 검색이 성공했다는 것은 다릅니다. 그래서 거리 값을 함께 봐야 합니다.

평소보다 훨씬 먼 것만 나왔다면 사실상 못 찾은 것입니다.

임베딩 모델을 중간에 바꾸면 어떻게 되는가

전부 다시 적재해야 합니다.

적재할 때 쓴 모델과 검색할 때 쓰는 모델이 다르면 좌표계가 다른 두 지도를 겹쳐 보는 셈이 됩니다.

무서운 것은 오류가 안 난다는 점입니다. 그냥 엉뚱한 결과가 조용히 나옵니다. `collection_name`과 `persist_directory`를 전 과정에서 고정한 것도 같은 이유입니다.

State는 그냥 변수와 무엇이 다른가

노드 사이를 오가는 유일한 통로라는 점이 다릅니다.

노드는 서로를 직접 부르지 않습니다. State에 쓰고 다음 노드가 State에서 읽습니다.

그래서 노드를 떼어내 따로 테스트할 수 있고 중간에 멈췄다가 이어서 시작할 수 있습니다.

전역 변수로 짜면 누가 언제 무엇을 바꿨는지 추적할 수 없습니다.

Reducer는 왜 필요한가

노드가 State에 값을 쓸 때 덮어쓸지 합칠지를 정해야 하기 때문입니다.

메시지 목록은 덮어쓰면 안 됩니다. 대화가 사라집니다. 그래서 `add_messages`를 씁니다.

그런데 이것이 단순히 뒤에 붙이기만 하는 것이 아닙니다. id를 보고 합칩니다.

아래 코드를 `demo_reducer.py`로 저장하고 실행해 직접 확인합니다.

모델을 부르지 않으므로 1초 안에 끝납니다.

```
# demo_reducer.py — add_messages 가 id 로 합친다는 것을 눈으로 보기
from langchain_core.messages import AIMessage
from langgraph.graph.message import add_messages

기존 = [AIMessage(content="첫 답변", id="m1")]

print("=== 새 id 로 넣으면 ===")
print([m.content for m in add_messages(기존, [AIMessage(content="두 번째", id="m
2")])])

print("\n=== 같은 id 로 넣으면 ===")
print([m.content for m in add_messages(기존, [AIMessage(content="고친 답변", id="m
1")])])
```

출력은 이렇게 나옵니다.

```
=== 새 id 로 넣으면 ===
['첫 답변', '두 번째']

=== 같은 id 로 넣으면 ===
['고친 답변']
```

이것을 알아야 5일차 마스킹 미들웨어가 이해됩니다. 원본 메시지를 가린 버전으로 바꾸려면 같은 id로 넣어야 합니다. 새 id로 넣으면 원본이 그대로 남아 마스킹이 무의미해집니다.

ReAct 루프가 멈추지 않을 때

모델이 도구 호출을 계속 요청하기 때문입니다. 도구가 원하는 답을 못 주면 조건을 바꿔 다시 부르고 그것이 반복됩니다.

`recursion_limit`으로 상한을 겁니다. 다만 상한은 안전장치이지 해결책이 아닙니다. 진짜 원인은 대개 셋 중 하나입니다. 도구 설명이 모호해서 모델이 잘못된 도구를 고르거나, 도구가 빈 결과를 주면서 왜 비었는지 알려주지 않거나, 종료 조건이 프롬프트에 없는 경우입니다.

도구를 부르지 않고 답을 지어낼 때

docstring이 곧 도구 설명입니다. 모델은 함수 이름과 docstring만 보고 언제 쓸지 정합니다.

"직원을 조회한다"보다 "직원 이름으로 소속 팀과 이메일을 조회한다. 사내 임직원 정보를 물으면 반드시 이 도구를 쓴다"가 훨씬 잘 불립니다. 도구를 안 부르면 코드를 고치기 전에 docstring부터 고쳐 보는 게 좋습니다. 대부분 거기서 해결됩니다.